

Programming and Algorithms

Aula 11: Tratamento de exceções em Python

Prof. Ettore Baldini-Neto
ettore.neto@faculdadeimpacta.com.br

Analogia: O elevador

- Imagine um elevador sem sensor de porta:
  - Alguém coloca a mão → elevador quebra ❌
- Agora com sensor de porta (tratamento):

  Alguém coloca a mão → porta reabre ✅

💡 **Exceções são os 'sensores' do seu programa!**

💡 **Elas detectam problemas e permitem recuperação**

O que são exceções?

Exceção é um evento que **interrompe** o fluxo normal

- Exemplos comuns:
  - ValueError : valor inválido (ex: letra onde se espera um número)
  - ZeroDivisionError : divisão por zero
  - FileNotFoundError : arquivo não existe
  - TypeError : operação com tipo errado
  - IndexError -:índice fora da lista
- Sem tratamento: programa para abruptamente

Hierarquia

Em Python, as exceções são organizadas em uma estrutura hierárquica baseada em classes.

Isso significa que as exceções são organizadas em uma hierarquia: cada exceção específica é uma subclasse de uma exceção mais geral. Por exemplo, ZeroDivisionError é um tipo de ArithmeticError, que por sua vez é um tipo de Exception.

Compreender essa árvore é fundamental para capturar os erros corretos nos seus blocos try...except.

Hierarquia

1. A Raiz: BaseException

É a classe base absoluta. Absolutamente todas as exceções em Python herdam dela. No entanto, como desenvolvedor, você **não deve** herdar suas próprias exceções dessa classe, nem tentar capturá-la diretamente, pois isso pode impedir o encerramento correto do seu programa.

Hirarquia

2. Exceções de Sistema (Filhas de BaseException)

Estas exceções herdam diretamente da raiz e existem fora do fluxo normal de erros de código.

Elas são projetadas assim para que um bloco genérico de captura de erros não as intercepte acidentalmente.

SystemExit: Levantada pela função sys.exit(). Serve para encerrar o programa.

KeyboardInterrupt: Levantada quando o usuário pressiona Ctrl+C no terminal para interromper a execução.

Hierarquia

3. O Tronco Principal: Exception

É a classe base para **todas as exceções comuns**, erros de lógica do programa e exceções integradas.

Sempre que você for criar uma exceção personalizada (customizada) para o seu código, ela deve herdar de Exception.

Quando se usa um except Exception as e: ocorre a captura de qualquer erro de código, mas ainda permite-se que KeyboardInterrupt encerre o programa normalmente.

Hierarquia

4. Ramificações Comuns (Filhas de Exception)

A partir de **Exception**, a árvore se divide em várias categorias específicas de erros. Aqui estão as mais comuns:

ArithmeticError: Base para todos os erros matemáticos.

Exemplos*:* ZeroDivisionError (divisão por zero), OverflowError.

LookupError: Base para erros ao buscar itens em coleções (listas, dicionários).

Exemplos: IndexError (tentar acessar um índice que não existe numa lista), KeyError (buscar uma chave inexistente num dicionário).

Hierarquia

TypeError: Ocorre quando uma operação ou função é aplicada a um objeto de tipo inadequado (ex: tentar somar um número com um texto).

ValueError: Ocorre quando uma operação recebe um argumento com o tipo certo, mas um valor inapropriado (ex: int('texto')).

NameError: Ocorre quando você tenta usar uma variável ou função que não foi definida.

Dica de ouro

Em um bloco try...except com múltiplos except, Python verifica a hierarquia de cima para baixo. Portanto, você deve sempre colocar os tratamentos de **exceções mais específicas (ex: KeyError) primeiro**, e as **mais genéricas (ex: Exception) por último**.

Exemplo

```python
def dividir_numeros():
    try:
        # Tenta converter a entrada do usuário para inteiro.
        # Pode gerar um ValueError se o usuário digitar letras ou símbolos.
        numerador = int(input("Digite o numerador: "))
        denominador = int(input("Digite o denominador: "))

        # Tenta realizar a divisão.
        # Pode gerar um ZeroDivisionError se o denominador for 0.
        resultado = numerador / denominador

        print(f"O resultado da divisão é: {resultado}")

    # 1. Tratamento MAIS ESPECÍFICO (Filhas da ArithmeticError e ValueError)
    except ValueError:
        print("Erro de Valor: Você deve digitar números inteiros válidos, não letras!")

    except ZeroDivisionError:
        print("Erro Matemático: Não é possível dividir um número por zero!")

    # 2. Tratamento GENÉRICO (O Tronco Principal)
    except Exception as e:
        # Isso vai capturar qualquer outro erro de código que não prevemos acima
        # (ex: um NameError ou TypeError não esperado).
        print(f"Ocorreu um erro inesperado: {e}")

# Executa o programa
print("--- Calculadora de Divisão ---")
dividir_numeros()
```

Principais blocos

• **try**: Bloco onde você coloca o código que pode gerar uma exceção (erro).

• **except**: Bloco executado **se** uma exceção ocorrer dentro do try. Pode capturar todos os erros ou tipos específicos.

• **else**: Bloco executado **se nenhuma exceção ocorrer** no try. Útil para código que depende do sucesso do try.

• **finally**: Bloco executado **sempre**, ocorrendo exceção ou não. Usado para ações obrigatórias (fechar arquivos, liberar recursos).

Dividindo dois números

```
def dividir(a, b):
    try:
        resultado = a / b
    except ZeroDivisionError:
        print("Erro: divisão por zero!")
    else:
        print(f"Divisão bem-sucedida: {resultado}")
    finally:
        print("Operação finalizada.\n")

# Testes
dividir(10, 2)
dividir(10, 0)
```

Convertendo número com else e finally

```python
def converter_para_int(valor):
    try:
        num = int(valor)
    except ValueError:
        print(f"Erro: '{valor}' não é um número inteiro válido.")
    else:
        print(f"Conversão bem-sucedida! Número: {num}")
    finally:
        print("Tentativa de conversão encerrada.\n")

# Testes
converter_para_int("42")
converter_para_int("abc")
```

Lendo arquivos

```python
def ler_arquivo(nome_arquivo):
    arquivo = None
    try:
        arquivo = open(nome_arquivo, 'r')
        conteudo = arquivo.read()
    except FileNotFoundError:
        print(f"Erro: arquivo '{nome_arquivo}' não encontrado.")
    else:
        print(f"Conteúdo do arquivo:\n{conteudo}")
    finally:
        if arquivo:
            arquivo.close()
            print("Arquivo fechado com sucesso.")
        print("Operação de leitura finalizada.\n")

# Testes
ler_arquivo("existe.txt")   # Crie um arquivo 'existe.txt' com algum texto
ler_arquivo("nao_existe.txt")
```

Calculadora segura: mais de um except

```
def calculadora_segura():
    while True:
        try:
            n1 = float(input('N1: '))
            n2 = float(input('N2: '))
            op = input('+ - * / ou 0-sair: ')
            if op == '0': break

            if op == '+': print(n1 + n2)
            elif op == '-': print(n1 - n2)
            elif op == '*': print(n1 * n2)
            elif op == '/': print(n1 / n2)
            else: print('Operação inválida')

        except ValueError:
            print('Digite números!')
        except ZeroDivisionError:
            print('Não divide por zero!')
```

✅ Programa NUNCA quebra!

Crie um programa que:

1. Peça dois números ao usuário
2. Calcule e exiba a divisão
3. Trate ValueError (entrada não numérica)
4. Trate ZeroDivisionError

Use um loop para validação da entrada, ou seja, até o usuário acertar a entrada

Solução:

```
while True:
    try:
        a = float(input("Digite o primeiro número: ").replace(",", "."))
        b = float(input("Digite o segundo número: ").replace(",", "."))
        resultado = a / b
    except ValueError:
        print("Entrada inválida: digite apenas números.")
    except ZeroDivisionError:
        print("Erro: não é possível dividir por zero.")
    else:
        print(f"{a} / {b} = {resultado}")
        break
```